



HTML, JavaScript, Spring Boot, H2 Database 연결

Spring Boot + H2 미니 게시판 흐름



목차

01

프로젝트 목표 및 CRUD

HTML, JavaScript, Spring Boot, H2를 연결한 게시판 만들기

02

프로젝트 전체 구조

HTML, JavaScript, Spring Boot, Repository, H2의 역할과 전체 흐름

03

static HTML 방식

static 폴더 구조와 JavaScript를 통한 데이터 처리 방식

04

REST API

데이터를 주고받는 서버 주소와 HTTP 메서드

05

Controller와 Repository

Controller의 요청 처리와 Repository의 DB 작업

06

CRUD 흐름 및 확인 도구

글쓰기, 목록, 상세보기, 수정, 삭제 흐름과 확인 도구

이번 프로젝트의 목표



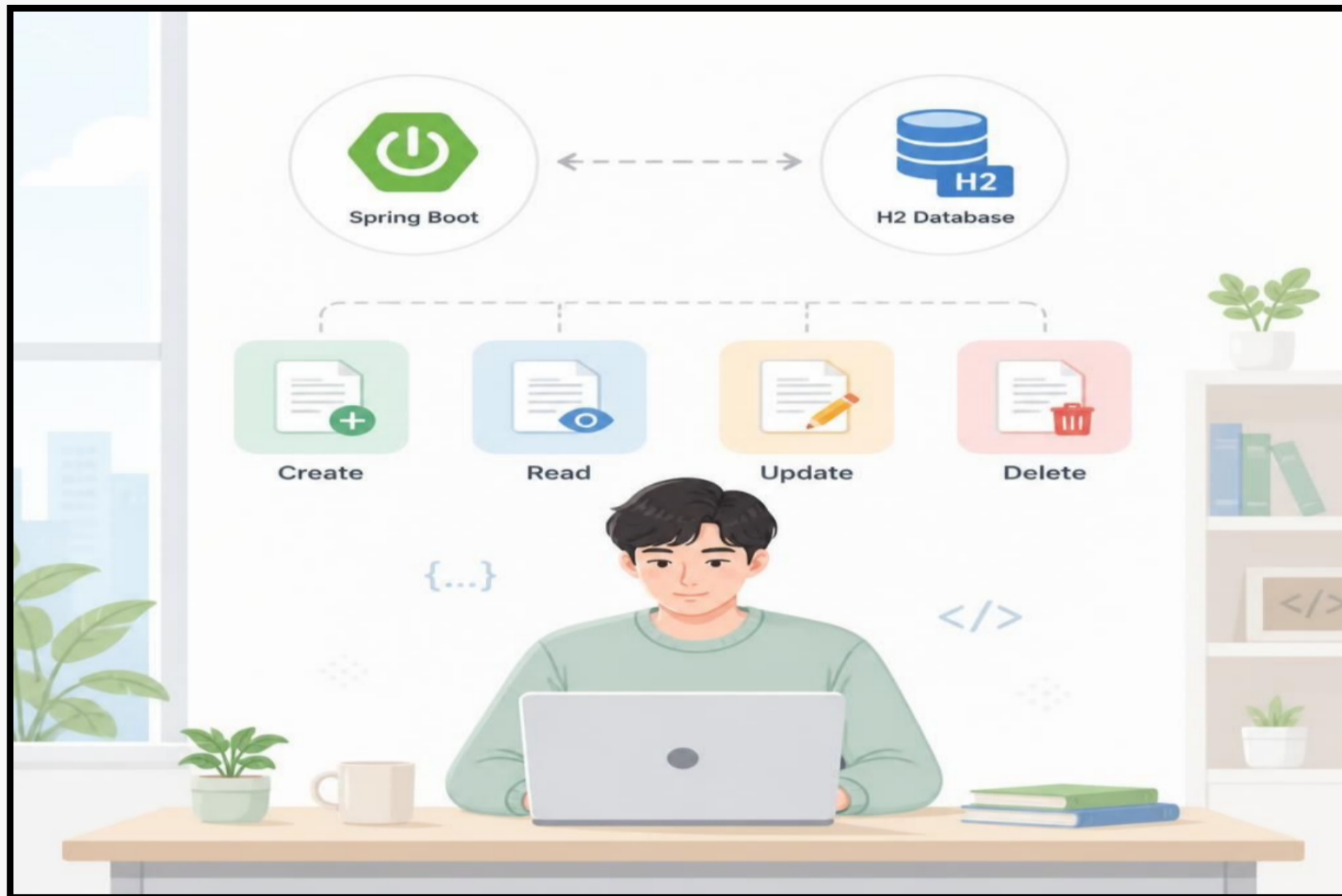
■

HTML, JavaScript, Spring Boot, H2 Database를 연결하여 간단한 게시판 CRUD 기능을 만든다.

HTML, JavaScript, Spring Boot, H2 Database를 연결하여 간단한 게시판 CRUD 기능을 만든다.

CRUD란 게시글을 만들고, 조회하고, 수정하고, 삭제하는 기본 기능이다.

CRUD란 게시글을 만들고, 조회하고, 수정하고, 삭제하는 기본 기능이다.





프로젝트 전체 구조

01

화면은 HTML이 담당한다.

화면은 HTML이 담당한다.

전체 흐름: HTML → JS fetch() → Spring REST →
Repository → H2 → JSON → 화면

02

데이터 요청은 JavaScript의 fetch()가 담당한다.

데이터 요청은 JavaScript의 fetch()가 담당한다.

전체 흐름: HTML → JS fetch() → Spring REST →
Repository → H2 → JSON → 화면

03

서버 처리는 Spring Boot가 담당한다.

서버 처리는 Spring Boot가 담당한다.

전체 흐름: HTML → JS fetch() → Spring REST →
Repository → H2 → JSON → 화면

04

데이터 저장은 H2 Database가 담당한다.

데이터 저장은 H2 Database가 담당한다.

전체 흐름: HTML → JS fetch() → Spring REST →
Repository → H2 → JSON → 화면



+

static HTML 방식

01

HTML 파일은 static 폴더에 둔다.
HTML 파일은 static 폴더에 둔다.

02

예시

- static/index.html
- static/board/list.html
- static/board/write.html
- static/board/detail.html

03

HTML은 화면을 보여주고, 데이터는
JavaScript로 서버에서 가져온다.
HTML은 화면을 보여주고, 데이터
는 JavaScript로 서버에서 가져온
다.



+

REST API란?



REST API는 화면이 아니라 데이터를 주고받는 서버 주소이다.

REST API는 화면이 아니라 데이터를 주고받는 서버 주소이다.

+

GET 요청

- GET /api/boards → 게시글 목록 조회
- GET /api/boards/1 → 1번 게시글 상세 조회

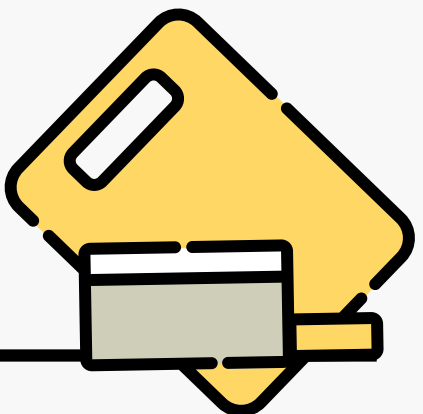
+

POST, PUT 요청

- POST /api/boards → 게시글 저장
- PUT /api/boards/1 → 1번 게시글 수정

DELETE 요청

- DELETE /api/boards/1 → 1번 게시글 삭제



Controller의 역할

요청 수신

Controller는 브라우저의 요청을 받는다.

@RestController는 HTML 화면이 아니라 JSON 데이터를 응답한다.

JSON 응답

처리 흐름

fetch(/api/boards)
→ BoardController
→ BoardRepository
→ H2 Database

Controller는 요청을 받고,
Repository는 실제 DB 작업을 처리한다.

JSON 형식으로 데이터를 반환한다.

Repository의 역할

01



findAll()

전체 게시글 조회

DB에서 모든
게시글을
가져온다.

02

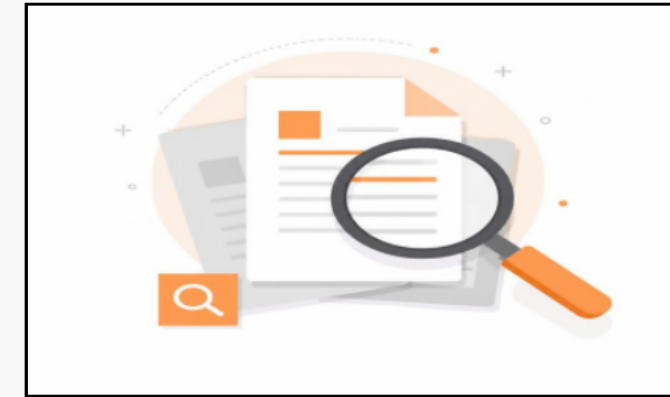


save()

게시글 저장
또는 수정

새 글 등록
또는 기존 글
업데이트

03



findById()

특정 게시글
조회

ID로 하나의
게시글을
찾는다.

04



deleteById()

게시글 삭제

ID로 해당
게시글을
DB에서
제거한다.



글쓰기 흐름

01

사용자 입력

사용자가 글쓰기 페이지에서 제목, 작성자, 내용을 입력한다.

등록 버튼을 클릭하여 작성을 완료한다.

02

서버 처리 과정

- 등록 버튼 클릭
- JavaScript createPost() 실행
- fetch()로 POST 요청
- Spring Controller의 save() 실행
- Repository가 DB에 저장
- H2에 INSERT 실행
- 목록 페이지로 이동

03

확인 방법

- 브라우저 Console: console.log(postData)
- Spring Console: insert into board 확인
- H2 Console: SELECT * FROM BOARD;



목록 출력 흐름

페이지 로드

목록 페이지가 열리면 JavaScript가 서버에 목록을 요청한다.

list.html이 실행되면서 자동으로 데이터를 가져온다.

서버 요청

확인 도구

- Thunder Client: GET /api/boards
- 브라우저 Console: console.log(data)
- Spring Console: select SQL 확인

- list.html 실행
- fetch(/api/boards) GET 요청
- boardRepository.findAll()
- DB에서 전체 게시글 조회
- JSON 응답
- JavaScript가 table에 출력

+

상세보기 흐름

01

제목 클릭

목록에서 제목을 클릭하면 상세 페이지로 이동한다.

/board/detail.html?id=글번호 형태로 이동한다.

02

서버 처리 과정

- 제목 클릭
- /board/detail.html?id=글번호
- detail.html에서 id 값 추출
- fetch(/api/boards/ + id)
- 서버에서 해당 게시글 조회
- JSON 응답
- 상세 화면에 출력

03

핵심 포인트

URL의 id를 이용해 어떤 게시글을 볼지 결정한다.

id 파라미터가 게시글 식별의 핵심이다.



+

수정 흐름

수정 시작

상세 페이지에서 내용을 수정한 뒤 수정 버튼을 누른다.

기존 내용을 변경하고 저장한다.

서버 처리

확인 방법

- Thunder Client: PUT /api/boards/1
- Spring Console: update SQL 확인
- H2 Console: SELECT * FROM BOARD;

- 수정 버튼 클릭
- updateBoard() 실행
- fetch()로 PUT 요청
- Controller의 update() 실행
- 기존 게시글 조회
- 제목·작성자·내용 변경
- save()로 저장
- 목록으로 이동

+

삭제 흐름

01

삭제 시작

- 상세 페이지에서 삭제 버튼을 누른다.
- 삭제 버튼 클릭
- confirm()으로 확인
- fetch()로 DELETE 요청

02

삭제 처리 과정

- Controller의 delete() 실행
- Repository가 DB에서 삭제
- 목록 페이지로 이동

03

삭제 확인

- Thunder Client: DELETE
/api/boards/1
- Spring Console: delete SQL
확인
- H2 Console: SELECT *
FROM BOARD;



+

+

확인 도구 정리

Thunder Client

01

API가 정상 동작하는지 확인

H2 Console

02

DB에 데이터가 실제로 저장됐는지 확인

브라우저 Console

03

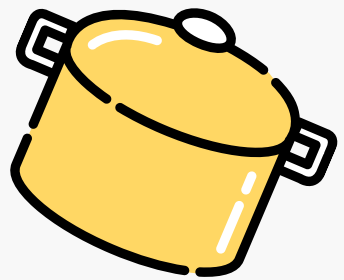
JavaScript 데이터 확인

Spring Console

04

SQL 실행 로그 확인

오늘의 핵심 정리



- HTML은 화면을 만든다. JavaScript는 서버에 요청한다.
- Spring Controller는 요청을 받는다. Repository는 DB 작업을 한다.
- H2는 데이터를 저장한다. JSON은 서버와 브라우저가 주고받는 데이터 형식이다.

+

■

최종 흐름:

화면 → 요청 → 서버 → DB → 응답 → 화면 갱신

+

자주 확인하면 어디에서 문제가 생겼는지 빠르게 찾을 수 있다.

■

